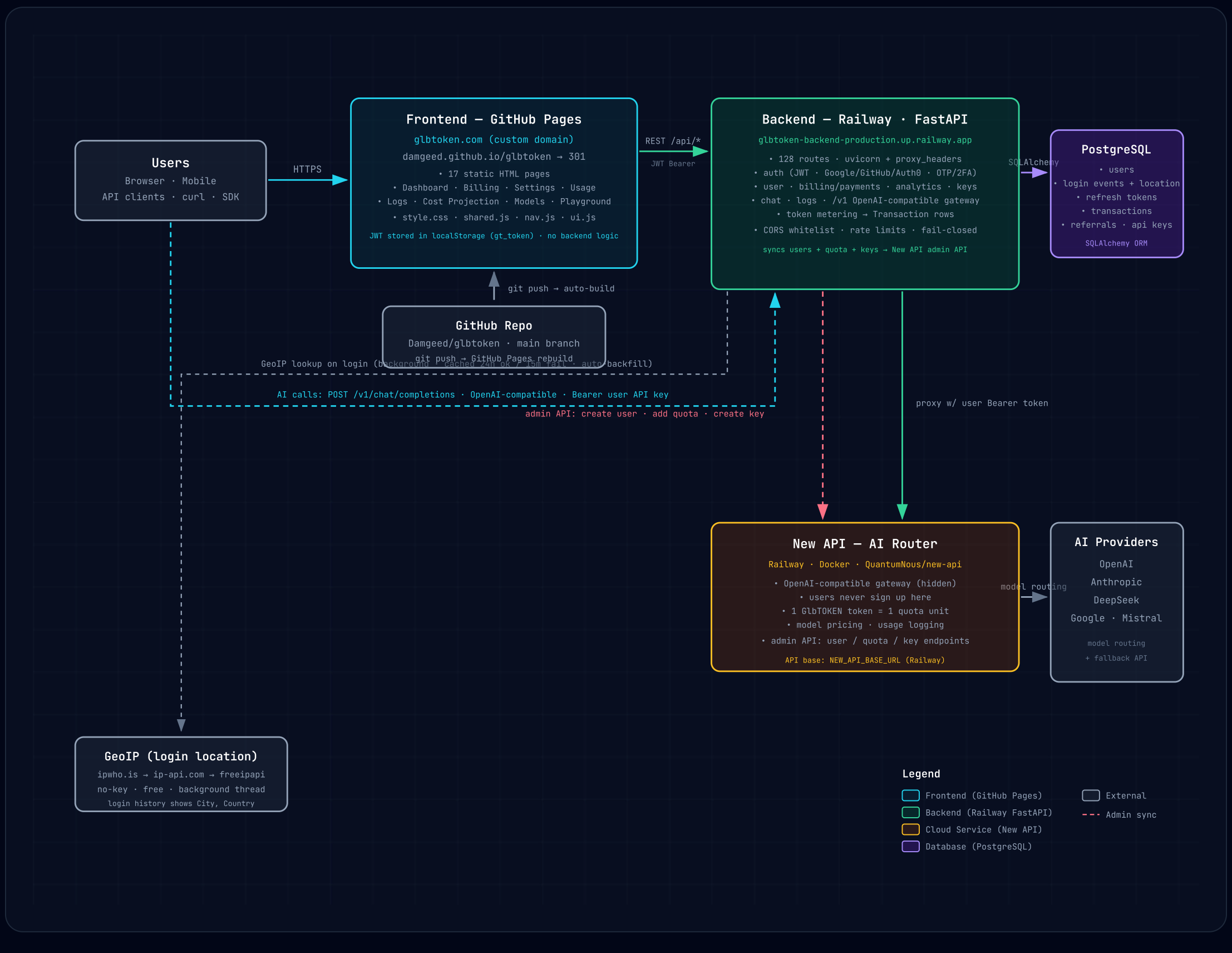
GlbTOKEN — System Architecture

GitHub Pages (frontend) → Railway FastAPI (backend + PostgreSQL) → New API (AI router) → upstream AI providers.
Users buy tokens on the frontend; the backend owns accounts/payments and syncs quota to New API; AI calls flow through an OpenAI-compatible gateway.



1 · Frontend — GitHub Pages

- Static site from repo **Damgeed/glbtoken**, main branch → Pages auto-build
- Served at **glbtoken.com** (301 from damgeed.github.io/glbtoken)
- 17+ pages: Dashboard, Billing, Settings, Usage, Logs, Cost Projection, Models, Playground…
- **No backend logic** — plain HTML/CSS/JS talking to Railway over REST
- Keeps JWT in localStorage (**gt_token**) and calls **glbtoken-backend-production.up.railway.app/api/***

2 · Backend — Railway · FastAPI

- Owns **accounts, payments, quota ledger, analytics** (128 routes)
- Auth: JWT + Google/GitHub/Auth0 OAuth + OTP/SMS 2FA; rate-limited, fail-closed
- **OpenAI-compatible /v1 gateway** — proxies AI calls to New API, meters tokens, writes Transaction rows
- PostgreSQL via SQLAlchemy: users, login events, refresh tokens, transactions, referrals
- Login location: **ipwho.is → ip-api.com → freeipapi** (15-min failure retry + auto-backfill)

3 · New API — AI Router

- **QuantumNous/new-api** on Railway (Docker) — the hidden AI gateway
- Users never register here; backend creates users + issues keys via **admin API**
- Quota sync: 1 GlbTOKEN token = 1 New API quota unit (add_quota on top-up)
- Routes model calls to **OpenAI · Anthropic · DeepSeek · Google · Mistral**
- Returns usage logs the backend mirrors into the user's dashboard